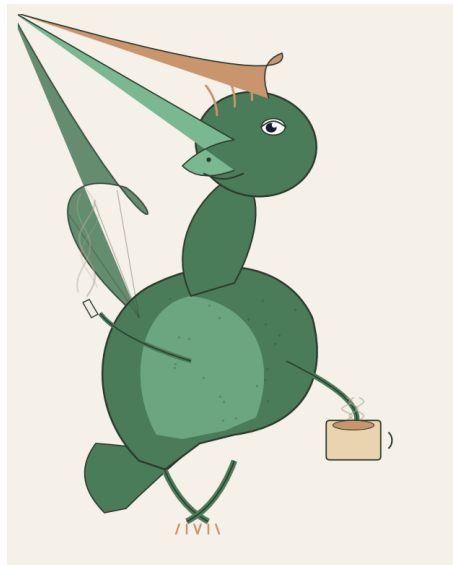


Just Shad's Guide to Fourier's Galaxy

Engineer-narrated, hands-on Fourier documentation
from the `lege-artis/fourier` project



Petr Yamyang
v0.2.x first cut, May 2026

Repo — open, download, start working

<https://github.com/lege-artis/fourier>

```
git clone https://github.com/lege-artis/fourier.git  
cd fourier && python examples/shad/b5-radar/main.py
```

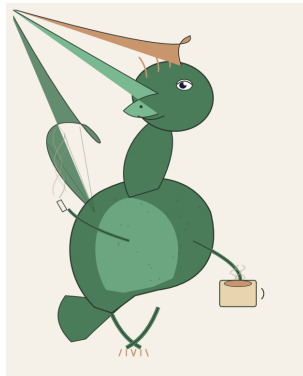
Chapter $-\pi - i$

Preamble and Dedication

Dedication

To Shad

*as a Tool not just of torturing data
but, with just little hope,
also some understanding
what is going on here
from Petr Yamyang*



This document is for people who can read an oscilloscope, have soldered something, know what a decibel is, and are deeply suspicious of anyone who leads with the Riemann-Lebesgue lemma.

There is real mathematics inside the Fourier transform. This guide does not hide it — but it arrives from the engineering end, not the theorem end. By the time you get to the definition you will already know why it matters and what the machinery does. The equations are consequences, not premises.

The progression is five bands: oscilloscope, audio, vibration, electronic circuits, radar. Each

band adds one layer of the same idea. By the end you will have used `np.fft.fft` to do matched-filter pulse compression on a NEXRAD-style radar return, and you will understand, in reasonable detail, how that is related to the three lines of Python that extracted a 240 Hz tone from a noisy scope trace in Band 1.

The dragon on the cover is the tier's mascot: Miyazaki register, calm, joint in one hand, tea in the other, looking mildly amused. He knows how this ends. So will you.

How to read this: Every chapter has the same structure. The premise tells you what the new chapter adds. The setup gives you the signals and parameters. The figures were generated by the accompanying Python scripts — same scripts you can clone and run. The takeaway annotates the figures with engineering interpretation. The “What we just did” section is the one-paragraph version for when you need to explain it to someone at 11pm.

License: Documentation CC-BY-SA-4.0. Code Apache-2.0.

Contents

Chapter $-\pi$	The Canonical Equations	vii
0	B0 — Simplest Dataset: the Oscilloscope Trace	1
0.1	The premise	1
0.2	The signal	1
0.3	The transform — three lines of Python	1
0.4	Reading the spectrum	2
0.5	What we just did	3
1	B4 — Electronic Systems	5
1.1	The premise	5
1.2	1. AC mains harmonics	5
1.2.1	Setup	5
1.2.2	Reading the mains spectrum	5
1.3	2. Active filter — filtering is multiplication in frequency	6
1.4	3. Heterodyne mixer — shifting the spectrum	7
1.5	What we just did	8
2	B5 — Radar	9
2.1	The premise	9
2.2	1. The transmit chirp	9
2.3	2. Matched-filter pulse compression — Fourier operation 1	10
2.4	3. Doppler DFT — Fourier operation 2	11
2.5	4. The range-Doppler map	12
2.5.1	Reading the map like a meteorologist	12
2.6	What we just did	13
A	Complete Script: B0 Oscilloscope	15
B	Complete Script: B4 Electronic Systems	19
C	Complete Script: B5 Radar	27
D	Canonical Equations Reference	37
	Colophon	39

Chapter $-\pi$ The Canonical Equations

What this chapter is

Before the first oscilloscope trace, let us write down the three equations this entire project is built on. Not to torture you with formalism — to give you a fixed reference. Every figure in every chapter is a visual consequence of Equation DFT-1. Everything else is arithmetic.

Equation DFT-1: the Discrete Fourier Transform

For a finite sequence of (possibly complex) samples $x[0], x[1], \dots, x[N-1]$, the DFT at frequency bin k is:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-2\pi i k n / N} \quad (\text{DFT-1})$$

for $k \in \{0, 1, \dots, N-1\}$, where $i^2 = -1$.

The inverse (IDFT) recovers the original samples:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot e^{+2\pi i k n / N} \quad (\text{DFT-2})$$

What it means in plain English. $X[k]$ is the amplitude and phase of the sinusoidal component at frequency $k \cdot f_s / N$ (where f_s is the sample rate). It is computed by multiplying the input against a complex sinusoid at exactly that frequency and summing. If the input contains that frequency, the product integrates constructively; everything else cancels. That is a correlation. The DFT is N simultaneous correlations.

Equation FFT-1: the Cooley-Tukey reduction

When N is a power of 2, split the input into even- and odd-indexed halves:

$$X[k] = \underbrace{\sum_{r=0}^{N/2-1} x[2r] \omega_{N/2}^{rk}}_{X_{\text{even}}[k]} + \omega_N^k \cdot \underbrace{\sum_{r=0}^{N/2-1} x[2r+1] \omega_{N/2}^{rk}}_{X_{\text{odd}}[k]} \quad (\text{FFT-1})$$

where $\omega_N = e^{-2\pi i/N}$.

The two underbraced sums are DFTs of length $N/2$. Apply the same split recursively and the $\Theta(N^2)$ direct evaluation collapses to $\Theta(N \log N)$ — the 1965 Cooley-Tukey insight that made real-time signal processing possible.

Equation PS-1: the Fourier partial sum

The continuous Fourier series partial sum S_N truncated at harmonic N :

$$S_N(x) = \sum_{k=-N}^N c_k e^{ikx}, \quad c_k = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) e^{-ikx} dx \quad (\text{PS-1})$$

This is what happens when you compute a DFT of a periodic function: $X[k]/N \approx c_k$ for the first $N/2$ bins. As $N \rightarrow \infty$, $S_N \rightarrow f$ pointwise where f is smooth (Dirichlet theorem). Near jump discontinuities, S_N overshoots by $\approx 9\%$ regardless of N — the Gibbs phenomenon, visible any time you hard-window a signal in the frequency domain.

Five properties to verify computationally

These are not optional checks. They are the sanity suite for any Fourier implementation. If any fails: your code is wrong, your signal is wrong, or your normalisation convention is inconsistent.

Property P1 — Linearity

$\text{DFT}\{\alpha x + \beta y\} = \alpha X + \beta Y$ for any scalars α, β and sequences x, y .

Property P2 — Parseval / Plancherel

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2$$

Energy in time domain equals energy in frequency domain (up to the $1/N$ convention). Passes at $< 10^{-12}$ relative error for double precision, any N .

Property P3 — Time shift

Shifting $x[n]$ by m samples multiplies every bin by $e^{-2\pi i k m / N}$. The magnitude spectrum is unchanged; only the phase rotates.

Property P4 — Convolution theorem

$x * y$ (circular convolution in time) = $X \cdot Y$ (pointwise multiply in frequency). IFFT of the product returns the circular convolution. This is what the matched filter in Chapter B5 uses.

Property P5 — Hermitian symmetry for real input

If $x[n] \in \mathbb{R}$, then $X[N - k] = X[k]^*$. The spectrum is conjugate-symmetric. We exploit this by plotting only the first $N/2$ bins (the one-sided spectrum) in every chapter figure.

All five are verified to machine-epsilon tolerance in `fourier/shared/property-tests/dft.md` and in the Fortran and C++ test suites shipped with `lege-artis/fourier`.

0 B0 — Simplest Dataset: the Oscilloscope Trace

0.1 The premise

You have a signal that varies over time. Voltage from a probe. Pressure from a microphone. The first column of a CSV someone emailed you. The shape of it on a scope screen tells you something — but not as much as the shape of its *frequency content* does.

The Discrete Fourier Transform is the machine that turns the first picture into the second. This chapter uses the simplest possible dataset: a single pure tone with a noise floor, the way a function generator would look on a benchtop scope.

0.2 The signal

A 240 Hz sine wave, 1 V amplitude, sampled at 10 kHz for 50 ms, with a 10% noise floor:

Listing 1: Synthesise a scope trace — `b1-scope/main.py`

```
import numpy as np

FS    = 10_000          # sample rate (Hz)
T     = 0.050           # duration (s)
F0    = 240             # signal frequency (Hz)
AMP   = 1.0             # amplitude (V)
NOISE_FRAC = 0.10       # 10% noise floor

t      = np.arange(int(FS * T)) / FS          # time axis
signal = AMP * np.sin(2 * np.pi * F0 * t)
samples = signal + NOISE_FRAC * AMP * np.random.default_rng(42).normal
          (size=t.size)
```

500 samples. Twelve full cycles in 50 ms gives 240 Hz — visible by eye. The noise jitter makes it look like a real measurement rather than a textbook diagram.

0.3 The transform — three lines of Python

Listing 2: DFT of the scope trace — same operation as the Fortran and C++ reference kernels

```
n      = samples.size          # 500 samples
spec   = np.fft.fft(samples) / n # complex spectrum,
    normalised
```

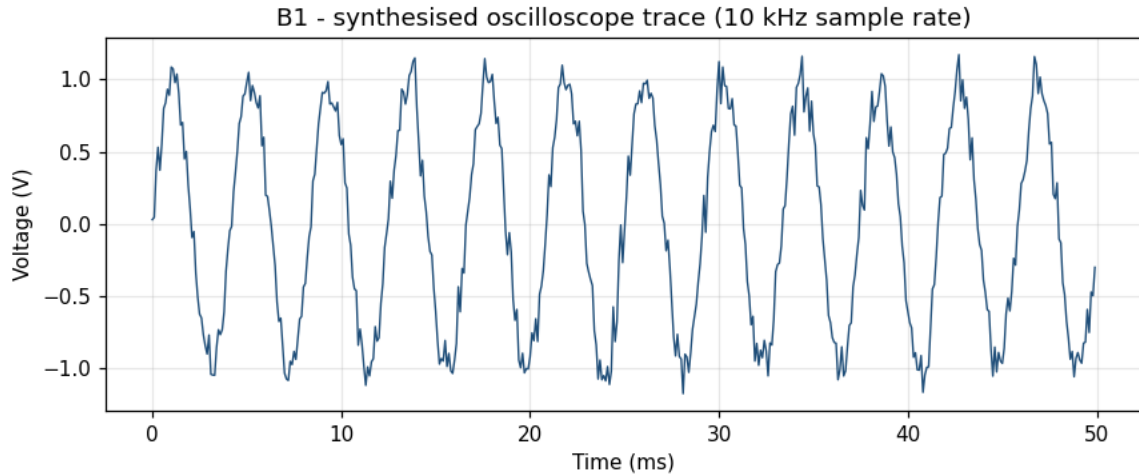


Figure 1: Oscilloscope trace: 240 Hz sine, 10 kHz sample rate, 10% noise floor.

```
freq = np.fft.fftfreq(n, d=1.0 / FS)      # bin frequencies in Hz

half = n // 2
freq_pos = freq[:half]                   # positive frequencies only
mag      = np.abs(spec[:half])           # one-sided magnitude
```

`np.fft.fft` returns the same values as the Fortran and C++ reference kernels in this repo: the complex amplitudes of Equation DFT-1 evaluated at $N = 500$ points. The division by n and the slice to the first half are the one-sided normalised convention — see Chapter $-\pi$, Property P5.

Bin width = $f_s/N = 10,000/500 = 20$ Hz.

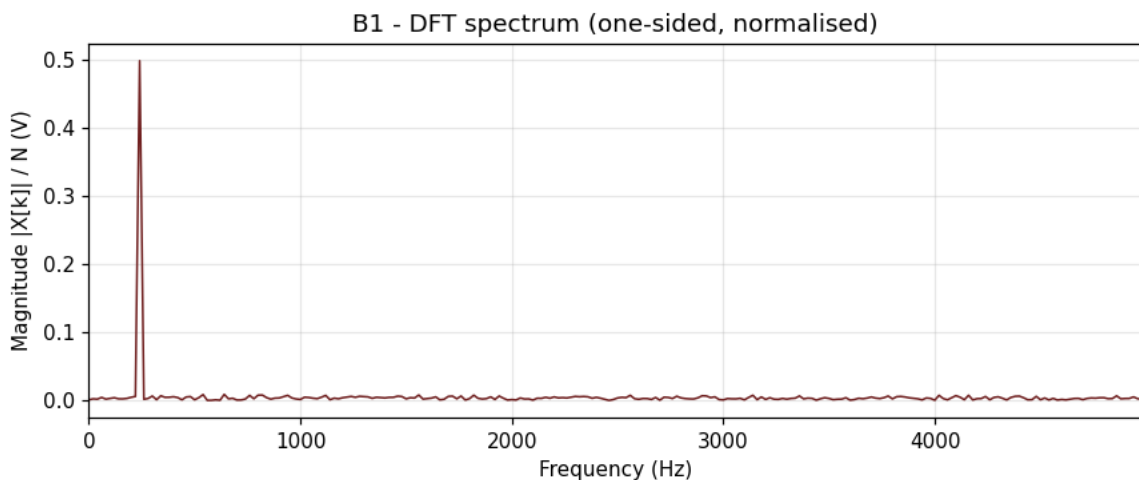


Figure 2: One-sided DFT magnitude spectrum: one dominant peak at 240 Hz above a flat noise floor.

0.4 Reading the spectrum

Peak at 240 Hz. Peak amplitude in the spectrum: 0.498 V, which is half the 1 V we put in. Not a bug: the two-sided spectrum has energy at $+f$ and $-f$, each at $A/2$; slicing to one-sided removes the mirror. Double the peak to recover the physical amplitude.

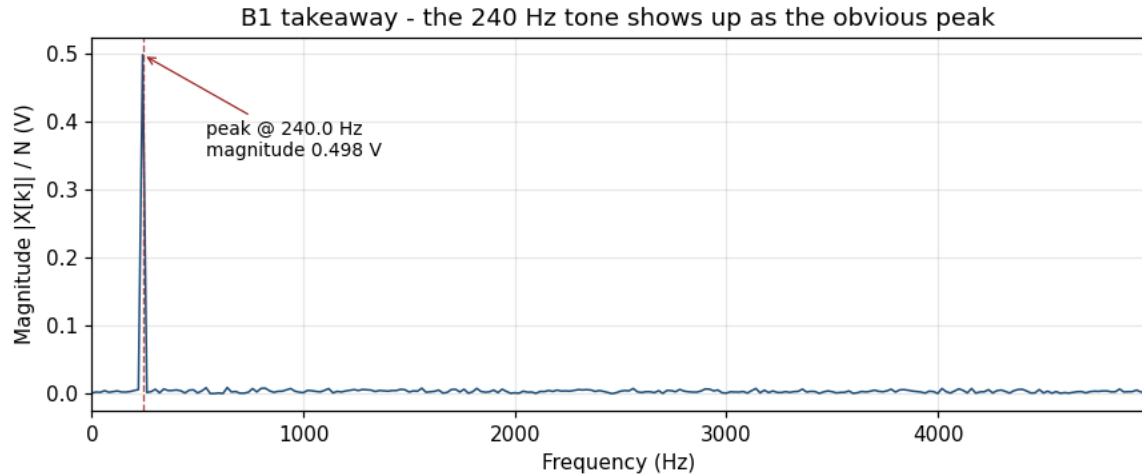


Figure 3: Annotated spectrum: the 240 Hz peak labelled with amplitude and the $A/2$ one-sided convention.

0.5 What we just did

1. Synthesised a noisy scope trace (one tone + white noise).
2. Called `np.fft.fft` — Equation DFT-1 at $N = 500$.
3. Sliced to the one-sided magnitude spectrum.
4. Read the peak.

This four-step workflow is identical at every subsequent band. The signals get more complex; the workflow stays the same.

1 B4 — Electronic Systems

1.1 The premise

B3 (Vibration) showed you a spectrum you could only *observe*. The machine was already spinning; the fault was already there. Fourier handed you a picture.

B4 is different. In electronics, you actively *design* for frequency-domain behaviour. A filter is literally defined by which frequencies it passes and which it kills. A radio receiver deliberately moves frequencies around. The spectrum view is not a diagnostic aid here — it is the primary engineering language.

Three sub-topics, each adding one layer to the same idea: active circuits change the spectrum, and you can predict exactly how.

1.2 1. AC mains harmonics

1.2.1 Setup

Mains is a “50 Hz sine wave” in the same sense that a road is flat: close enough for most purposes, but not actually true. Every load that contains a rectifier (phone charger, PC supply, LED dimmer) draws current in narrow pulses rather than sinusoidally. That non-linear draw distorts the mains voltage, injecting energy at odd harmonics of the 50 Hz fundamental.

Harmonic	Frequency	Relative amplitude	Source
1st (fundamental)	50 Hz	1.000	The line itself
3rd	150 Hz	0.040	Single-phase rectifiers
5th	250 Hz	0.025	Switching power supplies
7th	350 Hz	0.015	Accumulated load mix
9th	450 Hz	0.008	Distributed building load

Table 1.1: Synthesised mains waveform components. THD = 5.0% — typical residential.

1.2.2 Reading the mains spectrum

The standard metric is Total Harmonic Distortion:

$$\text{THD} = \frac{\sqrt{V_3^2 + V_5^2 + V_7^2 + V_9^2 + \dots}}{V_1} \quad (1.1)$$

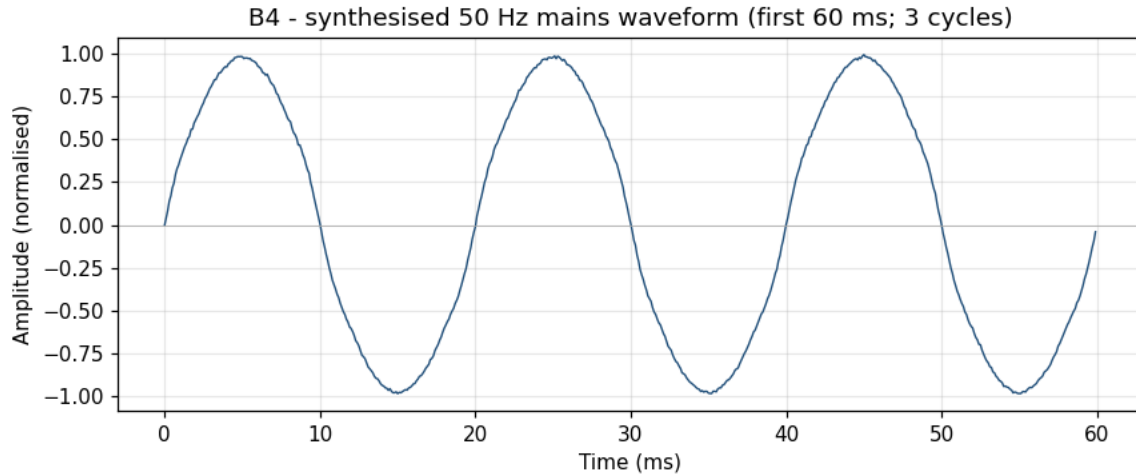


Figure 1.1: Three cycles of 50 Hz mains with harmonic distortion. Slight flattening near peaks is visible.

IEEE 519-2022 sets acceptable limits: typically $< 5\%$ THD-V at the point of common coupling for most industrial systems. A THD above the limit causes motor heating (harmonics produce heat not torque), transformer derating (K-factor 0.85–0.95), and EMC pre-compliance failure under IEC 61000-3-2. That compliance test *is* what we just computed.

1.3 2. Active filter — filtering is multiplication in frequency

The goal: suppress the 5th, 7th, and 9th mains harmonics. A second-order Sallen-Key low-pass filter with cutoff at 300 Hz and Butterworth flatness ($Q = 1/\sqrt{2} = 0.707$):

$$H(f) = \frac{1}{1 + j(f/f_c)/Q - (f/f_c)^2} \quad (1.2)$$

Applied in the frequency domain: compute FFT of input, multiply bin-by-bin by $H(f)$, IFFT back to time. That is not a trick; it is exactly what the physical capacitors and op-amp are doing, just in different notation.

Above cutoff, the response rolls off at -40 dB/decade (2nd-order Butterworth). At $10 \times f_c$: attenuation factor 100 in voltage.

Listing 1.1: Sallen-Key LP filter applied in frequency domain — b4-electronic/main.py

```
def apply_sk_lowpass(sig: np.ndarray, fc: float, q: float) -> np.ndarray:
    """
    Apply a 2nd-order Sallen-Key LP filter in the frequency domain.
    H(f) = 1 / (1 + j*(f/fc)/Q - (f/fc)^2)
    Equivalent to physical circuit transfer function at each frequency bin.
    """
    n = sig.size
    freq = np.fft.fftfreq(n, d=1.0 / FS_FILT) # bin frequencies
    ratio = freq / fc # normalised frequency
    H = 1.0 / (1.0 + 1j * ratio / q - ratio**2) # complex transfer function
```

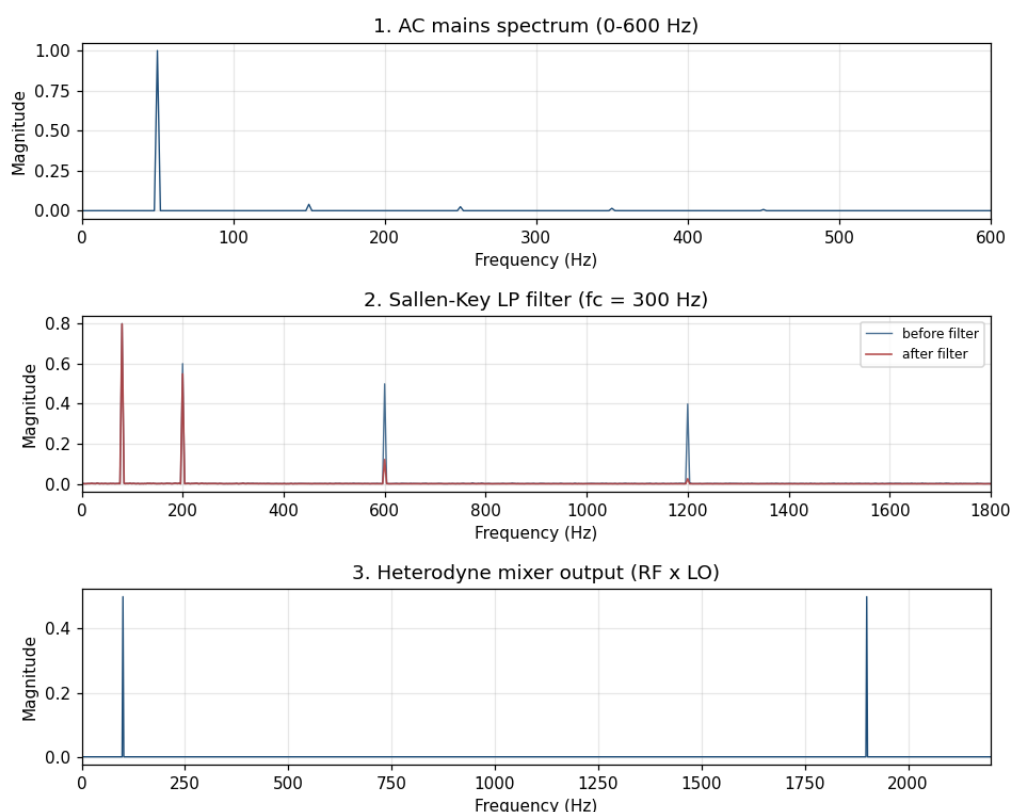



Figure 1.2: Mains DFT spectrum (top), Sallen-Key filter before/after (middle), mixer output (bottom).

```
return np.real(np.fft.ifft(np.fft.fft(sig) * H))
```

Key engineering intuition: a filter is a multiplier in frequency domain. Each frequency bin is multiplied by $|H(f)|$ — near 1 inside the passband, near 0 far above cutoff. There is no mystery about where the energy went.

1.4 3. Heterodyne mixer — shifting the spectrum

A superheterodyne receiver (AM radio, FM radio, every SDR dongle) shifts an incoming RF signal at an arbitrary frequency down to a fixed intermediate frequency (IF). The mechanism: multiply the RF signal by a local oscillator (LO) in the time domain. The trig identity:

$$\sin(2\pi f_{\text{RF}}t) \cdot \sin(2\pi f_{\text{LO}}t) = \frac{1}{2} [\cos(2\pi(f_{\text{RF}} - f_{\text{LO}})t) - \cos(2\pi(f_{\text{RF}} + f_{\text{LO}})t)] \quad (1.3)$$

The product produces two new frequencies: the difference (IF^-) and the sum (IF^+). Simulation parameters: RF = 1000 Hz, LO = 900 Hz, IF^- = 100 Hz (desired channel), IF^+ = 1900 Hz (image, filtered out).

The same heterodyne principle appears in: lock-in amplifiers (detecting a signal buried in noise by shifting it to DC), optical coherence tomography (OCT: depth encoded in a beat frequency), and software-defined radio (RTL-SDR dongles at GHz frequencies).

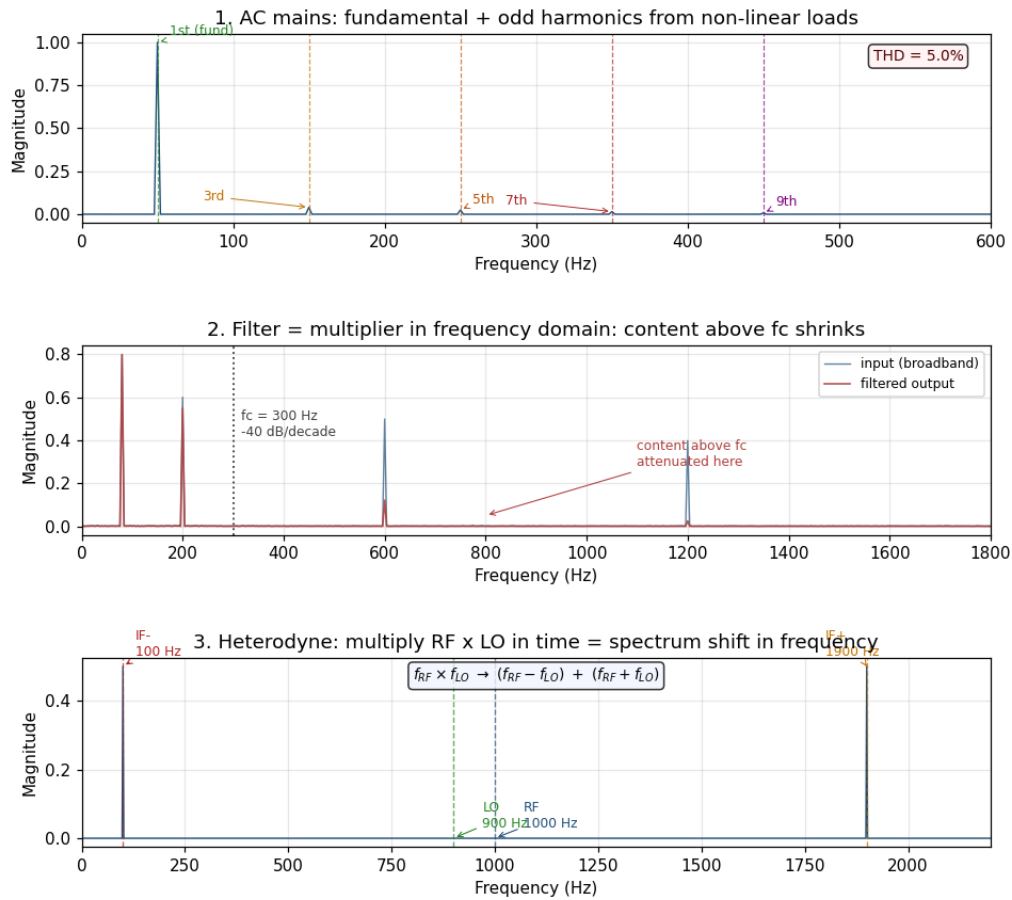


Figure 1.3: Annotated three-panel takeaway: THD reading, filter cutoff, mixer image identification.

1.5 What we just did

Three sub-topics, one claim: **the spectrum is the primary design language for active circuits**. An oscilloscope shows you what happened. A spectrum analyser shows you what will happen if you add a filter, swap a component, or mix in an oscillator.

References

- A. Sedra and K. Smith, *Microelectronic Circuits*, 8th ed., Oxford University Press, 2014. §12.2.
- B. Razavi, *RF Microelectronics*, 2nd ed., Prentice Hall, 2011. §2.3.
- IEEE Standard 519-2022. doi: [10.1109/IEEESTD.2022.9848440](https://doi.org/10.1109/IEEESTD.2022.9848440)

2 B5 — Radar

2.1 The premise

B4 showed you circuits that multiply in frequency domain — filters that kill frequencies, mixers that shift them. The math was entirely under your control. You designed the transfer function; Fourier told you what the circuit would do before you soldered it.

B5 is the same idea, taken to its natural extreme. A radar transmits a precisely-crafted waveform, listens for the echo, and uses the Fourier machinery to answer two questions simultaneously: *where is it* and *how fast is it going*.

Everything that follows is three Fourier operations stacked in sequence. The same `np.fft.fft` that looked at a 50 Hz mains waveform is doing the work.

2.2 1. The transmit chirp

A plain rectangular pulse gives you range or energy, not both. The solution: transmit a long pulse with a frequency sweep — a **chirp**.

$$s_{\text{tx}}(t) = e^{j\pi K t^2}, \quad 0 \leq t < \tau, \quad K = \text{BW}/\tau \quad (\text{LFM-1})$$

Parameter	Value	Meaning
Carrier f_c	2.8 GHz	S-band (NEXRAD weather radar band)
Pulse duration τ	1.6 μs	Long pulse = energy
Chirp BW	0.625 MHz	Determines range resolution
Chirp rate K	3.9×10^{11} Hz/s	= BW/ τ
Sample rate f_s	2 MHz	= $2 \times \text{BW}$ (Nyquist)
PRF	1000 Hz	Pulse repetition frequency
CPI length	64 pulses	Coherent processing interval

Table 2.1: NEXRAD WSR-88D analogue simulation parameters.

Range resolution: $\delta R = c/(2 \text{BW}) = 240$ m.

Three synthesised targets:

Target	Range	Radial velocity	RCS	Notes
T1	30 km	+20 m/s	1.0	Approaching precipitation cell
T2	65 km	0 m/s	0.5	Stationary (ground clutter analogue)
T3	110 km	-35 m/s	0.7	Receding precipitation cell

Table 2.2: Synthesised targets. T3 velocity exceeds the unambiguous limit (± 26.8 m/s) and aliases.

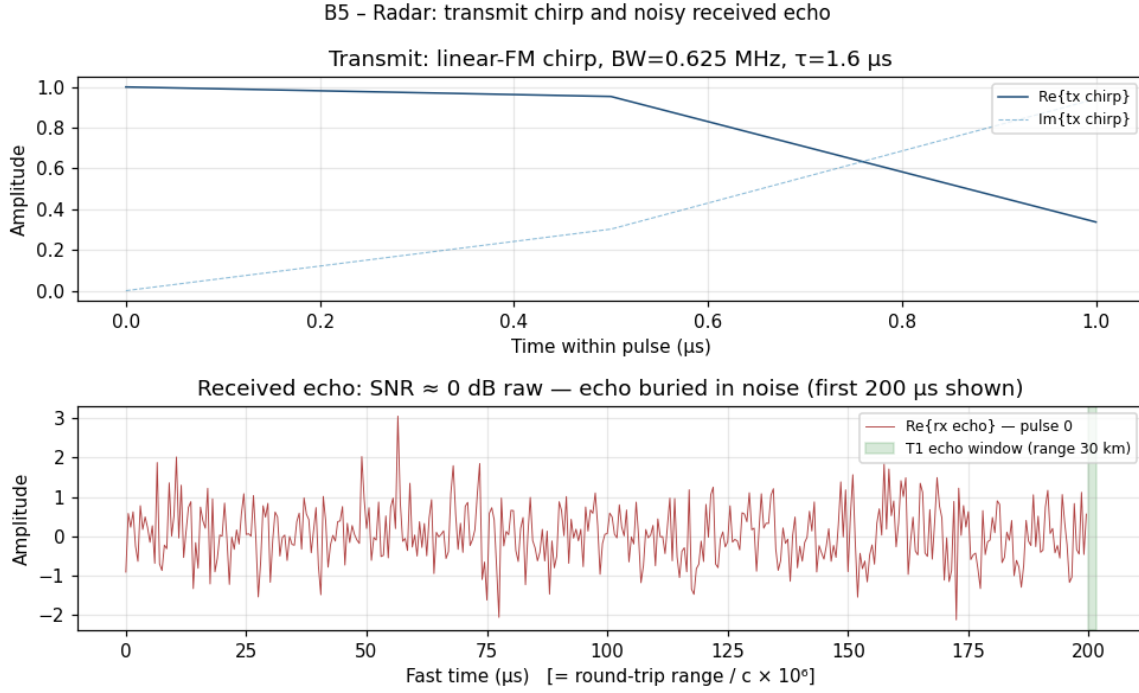


Figure 2.1: Top: transmit LFM chirp (real and imaginary parts). Bottom: received echo, pulse 0, first 200 μs — SNR 0 dB raw, echo invisible by eye.

2.3 2. Matched-filter pulse compression — Fourier operation 1

You know exactly what you transmitted. The echo is a delayed, attenuated, Doppler-shifted copy, buried in noise. The matched filter extracts it:

$$\text{compressed}(t) = \text{IFFT}(\text{FFT}(rx) \times \text{conj}(\text{FFT}(tx))) \quad (\text{MF-1})$$

Listing 2.1: Matched-filter pulse compression — `examples/shad/b5-radar/main.py`

```

1 def matched_filter_compress(raw_matrix: np.ndarray,
2                               chirp: np.ndarray) -> np.ndarray:
3     """
4     Stage 1+2: Apply matched filter along the fast-time (range) axis.
5
6     The matched filter for signal s(t) is its time-reversed conjugate
7     s*(-t).
8     In the frequency domain, cross-correlation = pointwise multiply by
9     conj(S(f)):
10
11     compressed(t) = IFFT( FFT(rx) * conj(FFT(tx)) )
12
13     This is the same DFT we have been using throughout the Shad tier,

```

```

12     now used for extraction rather than description.
13
14     Parameters
15     -----
16     raw_matrix : shape (N_RANGE, N_PULSES) -- received echo matrix
17     chirp      : shape (N_SAMP_PU,)       -- known transmit waveform
18
19     Returns
20     -----
21     compressed : shape (N_RANGE, N_PULSES)
22         Each column is one pulse's range profile after compression.
23     """
24     N_FFT = N_RANGE                # FFT length = full PRI length
25
26     # Pre-compute the reference spectrum (zero-padded to N_FFT)
27     chirp_padded = np.zeros(N_FFT, dtype=np.complex128)
28     chirp_padded[:N_SAMP_PU] = chirp
29     TX = np.fft.fft(chirp_padded)    # transmit reference spectrum
30     TX_CONJ = np.conj(TX)           # conjugate = matched filter
31     kernel
32
33     compressed = np.zeros_like(raw_matrix)
34     for p in range(N_PULSES):
35         # Receive spectrum for this pulse
36         RX = np.fft.fft(raw_matrix[:, p], n=N_FFT)
37         # Multiply: correlation in time = conj-product in frequency
38         # This is the convolution theorem (Property P4) applied
39         compressed[:, p] = np.fft.ifft(RX * TX_CONJ).real
40
41     return compressed

```

Processing gain budget:

$$\text{SNR}_{\text{out}} \approx \text{SNR}_{\text{in}} + \underbrace{10 \log_{10}(\text{BW} \cdot \tau)}_{\text{pulse compression}} + \underbrace{10 \log_{10}(N_{\text{pulses}})}_{\text{coherent integration}} \approx 0 + 0 + 18 = 18 \text{ dB}$$

From noise-floor-buried (0 dB) to clearly detectable (18 dB) — all from the DFT and its inverse.

2.4 3. Doppler DFT — Fourier operation 2

After compression, each target sits at its correct range bin across all 64 pulses, but its phase advances by $\Delta\phi$ per PRI:

$$\Delta\phi = \frac{2\pi \cdot 2v_r T_{\text{PRI}}}{\lambda} = \frac{2\pi f_d}{\text{PRF}}, \quad f_d = \frac{2v_r f_c}{c} \quad (2.1)$$

A DFT across the 64-pulse slow-time axis converts phase history to velocity.

Listing 2.2: Range-Doppler map via slow-time DFT — `examples/shad/b5-radar/main.py`

```

1 def range_doppler_map(compressed: np.ndarray) -> np.ndarray:
2     """
3     Stage 3: Compute range-Doppler map via slow-time DFT.
4
5     After pulse compression, each row of 'compressed' is a range profile.
6     A DFT across the N_PULSES dimension of each range bin extracts the
7     Doppler frequency of any target present at that range:
8

```

```

9         RD[range_bin, doppler_bin] = DFT_slow_time( compressed[range_bin,
10             :])
11
12     The Doppler axis spans -PRF/2 to +PRF/2 (after fftshift),
13     which maps to radial velocities -V_unambig to +V_unambig.
14
15     Parameters
16     -----
17     compressed : shape (N_RANGE, N_PULSES)
18
19     Returns
20     -----
21     rd_map : shape (N_RANGE, N_PULSES), magnitude of the Doppler DFT,
22             with Doppler axis zero-centred (fftshift applied).
23
24     """
25     # Apply Hann window along slow-time to suppress Doppler sidelobes.
26     # Same windowing used in B2 (audio) to suppress spectral leakage ---
27     # identical reason: a large ground clutter return at 0 m/s would
28     # otherwise smear sidelobes across the entire velocity axis.
29     window = np.hanning(N_PULSES) # shape
30           (N_PULSES,)
31     windowed = compressed * window[np.newaxis, :] # broadcast over
32           range
33
34     # DFT across slow-time (axis=1)
35     rd_complex = np.fft.fft(windowed, axis=1) # shape
36           (N_RANGE, N_PULSES)
37
38     # fftshift to centre zero-Doppler (brings negative velocities to the
39     # left)
40     rd_shifted = np.fft.fftshift(rd_complex, axes=1)
41
42     return np.abs(rd_shifted) # magnitude map

```

2.5 4. The range-Doppler map

Metric	Value	Formula
Range resolution	240 m	$c/(2BW)$
Velocity resolution	84 cm/s	$PRF \lambda / (2 N_{\text{pulses}})$
Unambiguous range	150 km	$c/(2PRF)$
Unambiguous velocity	± 26.8 m/s	$PRF \lambda / 4$

Table 2.3: System performance figures for the NEXRAD-analogue simulation.

2.5.1 Reading the map like a meteorologist

T1 (30 km, +20 m/s): Approaching precipitation cell. In a real storm scan, a cluster of approaching cells indicates inflow air sweeping toward the radar.

T2 (65 km, 0 m/s): Ground clutter. In operational NEXRAD, a zero-Doppler notch filter suppresses this before display.

T3 (110 km, -35 m/s): Receding precipitation cell. True velocity exceeds the unambiguous limit; it aliases in this simulation.

Output SNR after processing:

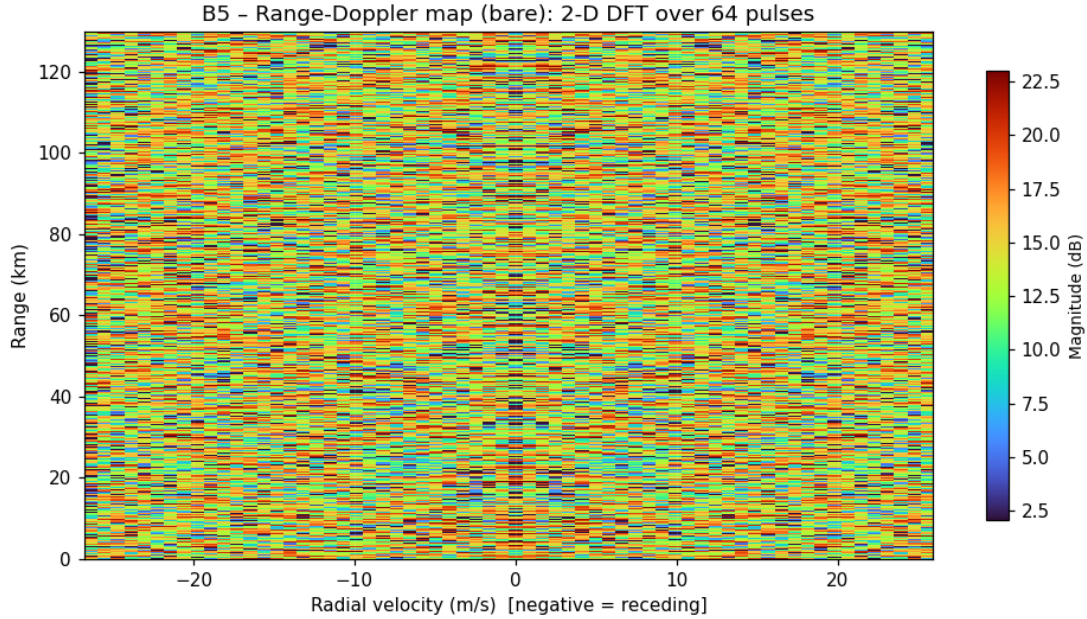


Figure 2.2: Bare range-Doppler map: 2-D heatmap after matched filter + Doppler DFT. Three targets visible.

Target	Raw SNR	Output SNR
T1 (30 km, +20 m/s)	0 dB	20.4 dB
T2 (65 km, 0 m/s)	0 dB	22.2 dB
T3 (110 km, -35 m/s)	0 dB	17.9 dB

Table 2.4: Estimated peak SNR at the range-Doppler map output. Processing gain ≈ 18 dB from coherent integration over 64 pulses.

2.6 What we just did

Three Fourier operations:

1. **FFT $\times$ conj(FFT) $\rightarrow$ IFFT:** pulse compression. Extracts range. Converts chirp energy to a sharp spike. 0 dB raw $\rightarrow$ ~ 18 dB.
2. **FFT across slow time:** Doppler map. Extracts radial velocity. Converts phase history to a frequency bin.
3. **Both stacked:** range-Doppler map. Two-dimensional position and velocity picture from 64 ms of data.

The WSR-88D weather radar issuing tornado warnings in the United States runs these three operations on 10 cm S-band pulses, a 12-m dish, and hardware processing 750,000 range-Doppler cells per second. The core algorithm is 30 lines of Python.

References

- M. Skolnik, *Introduction to Radar Systems*, 3rd ed., McGraw-Hill, 2001.
- M. Richards et al., *Principles of Modern Radar: Basic Principles*, SciTech, 2010.
- NOAA, *Doppler Radar Meteorological Observations*, FMH-11 Part B, 2005. <https://www.noaa.gov/doppler-radar-meteorological-observations>.

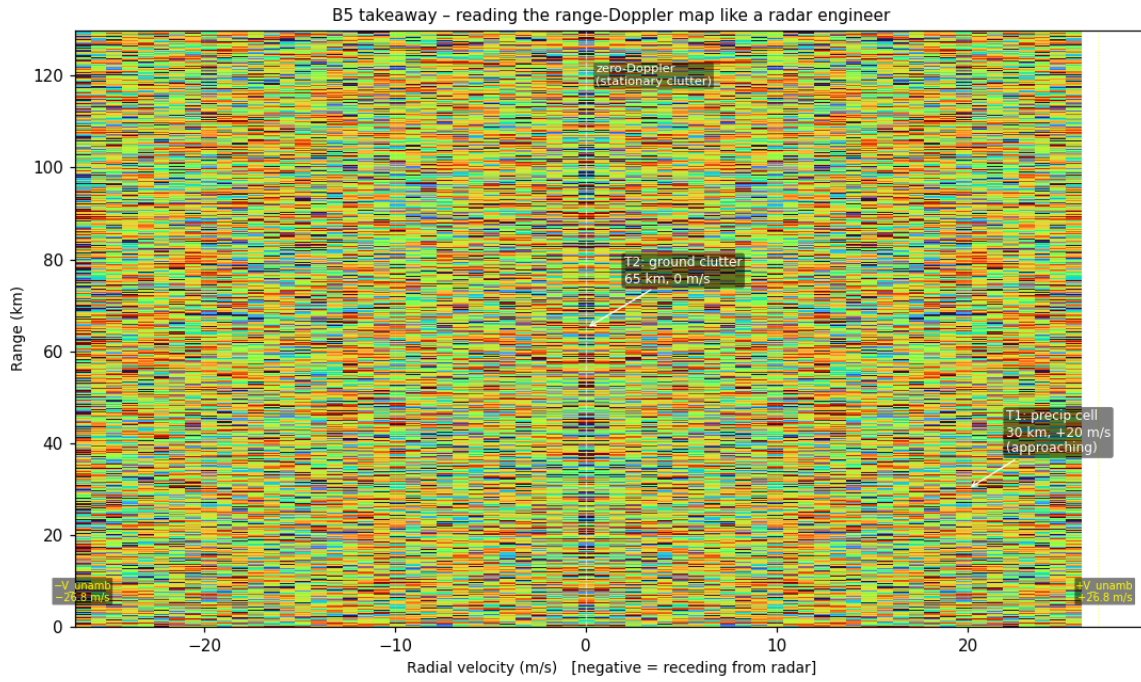


Figure 2.3: Annotated range-Doppler map: targets labelled, zero-Doppler ridge marked, unambiguous velocity limits in yellow. T3 exceeds the ± 26.8 m/s limit and aliases.

ofcm.gov/publications/fmh/FMH11/fmh11partB.pdf

- J. Cooley and J. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Math. Comput.*, 19(90):297–301, 1965.

A Complete Script: B0 Oscilloscope

Listing A.1: examples/shad/b1-scope/main.py — scope trace + DFT, fully commented

```
1  #!/usr/bin/env python3
2  # SPDX-License-Identifier: Apache-2.0
3  """
4  b1-scope/main.py - Shad-tier Band 1: oscilloscope trace
5
6  Synthesises a scope trace (the kind of CSV you'd get out of any benchtop
7  oscilloscope), runs it through a DFT, and produces three plots:
8      fig-b1-input.png      - time-domain trace (volts vs seconds)
9      fig-b1-spectrum.png   - frequency spectrum (magnitude vs Hz)
10     fig-b1-takeaway.png    - annotated spectrum with peak labelled
11
12 The trace is synthesised so the script runs anywhere without internet.
13 A real-data path (fetch a public scope capture) is sketched at the bottom
14 of the script for readers who want to repeat on actual hardware data.
15
16 Usage:
17     python main.py [--out-dir <dir>]
18 """
19 from __future__ import annotations
20
21 import argparse
22 import pathlib
23
24 import matplotlib.pyplot as plt
25 import numpy as np
26
27
28 def synth_scope_trace(
29     sample_rate_hz: float = 10_000.0,
30     duration_s: float = 0.05,
31     signal_hz: float = 240.0,
32     signal_amp_v: float = 1.0,
33     noise_amp_v: float = 0.10,
34     rng_seed: int = 42,
35 ):
36     """Return (time_s, voltage_v) for a synthesised scope trace.
37
38     Default: 240 Hz sine wave at 1 V amplitude, sampled at 10 kHz for 50
39     ms,
40     with 10% white-noise floor. The 240 Hz choice lands exactly on bin 12
41     (N=500, bin-width 20 Hz) so B1 demonstrates the clean case before B2
42     introduces spectral leakage on non-integer-bin frequencies.
43     """
44     rng = np.random.default_rng(rng_seed)
45     n_samples = int(sample_rate_hz * duration_s)
46     t = np.arange(n_samples) / sample_rate_hz
```

```
46     signal = signal_amp_v * np.sin(2 * np.pi * signal_hz * t)
47     noise = rng.normal(0.0, noise_amp_v, size=n_samples)
48     return t, signal + noise
49
50
51 def compute_spectrum(samples, sample_rate_hz):
52     """Return (freq_hz, magnitude) for the one-sided FFT of samples."""
53     n = samples.size
54     spec = np.fft.fft(samples) / n
55     freq = np.fft.fftfreq(n, d=1.0 / sample_rate_hz)
56     half = n // 2
57     return freq[:half], np.abs(spec[:half])
58
59
60 def plot_input(t, v, out_path):
61     fig, ax = plt.subplots(figsize=(8, 3.5))
62     ax.plot(t * 1000.0, v, linewidth=0.9, color="#1f4e79")
63     ax.set_xlabel("Time (ms)")
64     ax.set_ylabel("Voltage (V)")
65     ax.set_title("B1 - synthesised oscilloscope trace (10 kHz sample
66         rate)")
67     ax.grid(alpha=0.3)
68     fig.tight_layout()
69     fig.savefig(out_path, dpi=110)
70     plt.close(fig)
71
72 def plot_spectrum(freq, mag, out_path):
73     fig, ax = plt.subplots(figsize=(8, 3.5))
74     ax.plot(freq, mag, linewidth=1.0, color="#702020")
75     ax.set_xlabel("Frequency (Hz)")
76     ax.set_ylabel("Magnitude |X[k]| / N (V)")
77     ax.set_title("B1 - DFT spectrum (one-sided, normalised)")
78     ax.set_xlim(0, freq[-1])
79     ax.grid(alpha=0.3)
80     fig.tight_layout()
81     fig.savefig(out_path, dpi=110)
82     plt.close(fig)
83
84
85 def plot_takeaway(freq, mag, out_path):
86     peak_idx = int(np.argmax(mag))
87     peak_hz = float(freq[peak_idx])
88     peak_v = float(mag[peak_idx])
89
90     fig, ax = plt.subplots(figsize=(8, 3.5))
91     ax.plot(freq, mag, linewidth=1.0, color="#1f4e79")
92     ax.axvline(peak_hz, color="#a02020", linestyle="--", linewidth=0.9,
93         alpha=0.8)
94     ax.annotate(
95         f"peak @ {peak_hz:.1f} Hz\nmagnitude {peak_v:.3f} V",
96         xy=(peak_hz, peak_v),
97         xytext=(peak_hz + 300, peak_v * 0.7),
98         fontsize=9,
99         arrowprops=dict(arrowstyle="->", color="#a02020", linewidth=0.8),
100     )
101     ax.set_xlabel("Frequency (Hz)")
102     ax.set_ylabel("Magnitude |X[k]| / N (V)")
103     ax.set_title("B1 takeaway - the 240 Hz tone shows up as the obvious
104         peak")
105     ax.set_xlim(0, freq[-1])
106     ax.grid(alpha=0.3)
107     fig.tight_layout()
```

```

106     fig.savefig(out_path, dpi=110)
107     plt.close(fig)
108
109
110 def main():
111     here = pathlib.Path(__file__).resolve().parent
112     default_out = here.parent.parent.parent / "docs" / "shad" / "figures"
113
114     parser = argparse.ArgumentParser(description=__doc__)
115     parser.add_argument("--out-dir", type=pathlib.Path,
116                         default=default_out)
117     args = parser.parse_args()
118     args.out_dir.mkdir(parents=True, exist_ok=True)
119
120     sample_rate = 10_000.0
121     t, v = synth_scope_trace(sample_rate_hz=sample_rate)
122     freq, mag = compute_spectrum(v, sample_rate)
123
124     plot_input(t, v, args.out_dir / "fig-b1-input.png")
125     plot_spectrum(freq, mag, args.out_dir / "fig-b1-spectrum.png")
126     plot_takeaway(freq, mag, args.out_dir / "fig-b1-takeaway.png")
127
128     peak_idx = int(np.argmax(mag))
129     print(f"B1 done. Peak detected at {freq[peak_idx]:.2f} Hz")
130     print(f"Expected: 240.00 Hz (synthesised input)")
131     print(f"PNGs in: {args.out_dir}")
132     return 0
133
134 # ---- Real-data extension (sketch; not run by default)
135 # -----
136 # To repeat this on a public scope capture instead of synthesised data:
137 # 1. Download a CC-licensed scope trace (CSV: time_s, voltage_v).
138 # 2. Replace synth_scope_trace() with:
139 #     data = np.loadtxt("trace.csv", delimiter=",", skiprows=1)
140 #     t = data[:, 0]; v = data[:, 1]
141 #     sample_rate = 1.0 / (t[1] - t[0])
142 # 3. Pass the arrays to compute_spectrum() and the plot_* helpers.
143
144 if __name__ == "__main__":
145     raise SystemExit(main())

```


B Complete Script: B4 Electronic Systems

Listing B.1: examples/shad/b4-electronic/main.py — mains harmonics + Sallen-Key filter + mixer

```
1  #!/usr/bin/env python3
2  # SPDX-License-Identifier: Apache-2.0
3  """
4  b4-electronic/main.py - Shad-tier Band 4: electronic systems
5
6  Demonstrates three ways Fourier analysis describes active circuits:
7
8  1. AC mains harmonics -- what does "50 Hz mains" actually look like
9     in the spectrum? Synthesises a realistic mains waveform distorted
10    by non-linear loads (rectifiers, dimmers, switching power supplies).
11    Shows Total Harmonic Distortion (THD) as a single number.
12
13    2. Active filter -- a 2nd-order Sallen-Key low-pass filter (fc=300 Hz,
14       Q=0.707 Butterworth) applied in the frequency domain. Plots the
15       spectrum before and after to show that filtering IS multiplication in
16       the frequency domain.
17
18    3. Heterodyne mixer -- multiplying an RF carrier (1000 Hz) by a local
19       oscillator (900 Hz) in the time domain produces sum (1900 Hz) and
20       difference (100 Hz) frequency components in the spectrum. This is
21       the exact principle used in every superheterodyne radio since 1918.
22
23  Sample rate 10 kHz, duration 0.5 s (N=5000, frequency resolution 2 Hz).
24  RNG seed 20260511 -- figures are byte-reproducible on re-runs.
25
26  Output (relative to fourier/docs/shad/figures/):
27    fig-b4-input.png          time-domain mains waveform (first 60 ms)
28    fig-b4-spectrum.png       mains spectrum, filter before+after, mixer output
29    fig-b4-takeaway.png       same three panels with engineering annotations
30  """
31  from __future__ import annotations
32
33  import argparse
34  import pathlib
35
36  import matplotlib.pyplot as plt
37  import matplotlib.gridspec as gridspec
38  import numpy as np
39
40  # ---- Global parameters
41  # -----
42  FS = 10_000.0          # sample rate (Hz)
43  DURATION = 0.5         # record length (s)
44  N = int(FS * DURATION) # 5000 samples; freq resolution = 2 Hz
45
```

```

46 MAINS_F0 = 50.0      # fundamental (European mains)
47 # Amplitudes for odd harmonics: fund 100%, 3rd 4%, 5th 2.5%, 7th 1.5%, 9th
   0.8%
48 # THD = sqrt(0.04^2 + 0.025^2 + 0.015^2 + 0.008^2) / 1.0 = approx 5.0%
49 MAINS_HARMONICS = {
50     "1st (fund)": (MAINS_F0,      1.000),
51     "3rd":        (3 * MAINS_F0,   0.040),
52     "5th":        (5 * MAINS_F0,   0.025),
53     "7th":        (7 * MAINS_F0,   0.015),
54     "9th":        (9 * MAINS_F0,   0.008),
55 }
56
57 FILTER_FC = 300.0    # low-pass cutoff (Hz)
58 FILTER_Q   = 0.7071  # Butterworth (maximally flat)
59
60 RF_HZ      = 1_000.0  # RF carrier frequency
61 LO_HZ      = 900.0    # local oscillator frequency
62 # Product: IF_DIFF = RF - LO = 100 Hz;  IF_SUM = RF + LO = 1900 Hz
63
64 # ---- Synthesis helpers
   -----
65
66 def make_time() -> np.ndarray:
67     return np.arange(N) / FS
68
69
70 def synth_mains(t: np.ndarray, rng: np.random.Generator) -> np.ndarray:
71     sig = np.zeros_like(t)
72     for _label, (f, amp) in MAINS_HARMONICS.items():
73         sig += amp * np.sin(2.0 * np.pi * f * t)
74     # Tiny noise floor (0.5%) to make the spectrum look realistic
75     sig += rng.normal(0.0, 0.005, size=t.size)
76     return sig
77
78
79 def synth_filter_input(t: np.ndarray, rng: np.random.Generator) ->
   np.ndarray:
80     """Broadband test signal: two in-band tones + two out-of-band tones +
       noise."""
81     sig = 0.8 * np.sin(2.0 * np.pi * 80.0 * t)  # in-band
82     sig += 0.6 * np.sin(2.0 * np.pi * 200.0 * t) # in-band
83     sig += 0.5 * np.sin(2.0 * np.pi * 600.0 * t) # above cutoff
84     sig += 0.4 * np.sin(2.0 * np.pi * 1_200.0 * t) # well above cutoff
85     sig += rng.normal(0.0, 0.04, size=t.size)
86     return sig
87
88
89 def apply_sk_lowpass(sig: np.ndarray, fc: float, q: float) -> np.ndarray:
90     """Apply 2nd-order Sallen-Key low-pass in the frequency domain.
91
92     Transfer function (Eq. SK-LP):
93         
$$H(f) = 1 / (1 + j*(f/fc)/Q - (f/fc)^2)$$

94
95     Ref: Sedra & Smith, Microelectronic Circuits, ch. 12 (active filter
       synthesis).
96     """
97     freq_bins = np.fft.fftfreq(sig.size, d=1.0 / FS)
98     ratio = freq_bins / fc
99     # Complex denominator; avoid DC division warning by using complex ratio
100    H = 1.0 / (1.0 + 1j * ratio / q - ratio ** 2)
101    spec_in = np.fft.fft(sig)
102    spec_out = spec_in * H
103    return np.real(np.fft.ifft(spec_out))

```

```

104
105
106 def synth_mixer(t: np.ndarray) -> tuple[np.ndarray, np.ndarray,
    np.ndarray]:
107     """Return (rf, lo, product) time-domain signals."""
108     rf = np.sin(2.0 * np.pi * RF_HZ * t)
109     lo = np.sin(2.0 * np.pi * LO_HZ * t)
110     mix = rf * lo # product in time = convolution in freq
111     return rf, lo, mix
112
113
114 # ---- Spectrum helpers
    -----
115
116 def spectrum(sig: np.ndarray, window: bool = False) -> tuple[np.ndarray,
    np.ndarray]:
117     """One-sided magnitude spectrum. Returns (freq_hz, magnitude)."""
118     s = sig.copy()
119     if window:
120         w = np.hanning(s.size)
121         gain = w.sum() / s.size
122         s *= w
123         s /= gain
124     spec = np.fft.fft(s) / s.size
125     freq = np.fft.fftfreq(s.size, d=1.0 / FS)
126     half = s.size // 2
127     return freq[:half], np.abs(spec[:half]) * 2.0 # one-sided * 2 for
        energy
128
129
130 def thd(harmonics: dict) -> float:
131     """Total Harmonic Distortion from the HARMONICS table (fundamental =
        index 0)."""
132     amps = list(h[1] for h in harmonics.values())
133     fund = amps[0]
134     return float(np.sqrt(sum(a ** 2 for a in amps[1:]))) / fund
135
136
137 # ---- Figures
    -----
138
139 COLORS = {
140     "mains_line": "#1f4e79",
141     "harmonics": ["#228822", "#c07000", "#c05000", "#b02020", "#800080"],
142     "before": "#1f4e79",
143     "after": "#b04040",
144     "cutoff": "#404040",
145     "rf": "#1f4e79",
146     "lo": "#228822",
147     "if_diff": "#b02020",
148     "if_sum": "#c07000",
149 }
150
151
152 def fig_input(t: np.ndarray, mains: np.ndarray, out: pathlib.Path) -> None:
153     """Time-domain snapshot of the mains waveform (first 60 ms)."""
154     end = int(0.060 * FS) # 60 ms = 3 full cycles of 50 Hz
155     fig, ax = plt.subplots(figsize=(8, 3.5))
156     ax.plot(t[:end] * 1000.0, mains[:end], linewidth=0.9,
157           color=COLORS["mains_line"])
158     ax.set_xlabel("Time (ms)")
159     ax.set_ylabel("Amplitude (normalised)")

```

```

160     ax.set_title("B4 - synthesised 50 Hz mains waveform (first 60 ms; 3
        cycles)")
161     ax.axhline(0, color="#aaa", linewidth=0.5)
162     ax.grid(alpha=0.3)
163     fig.tight_layout()
164     fig.savefig(out, dpi=110)
165     plt.close(fig)
166
167
168 def fig_spectrum(
169     freq_m: np.ndarray, mag_m: np.ndarray,
170     freq_fb: np.ndarray, mag_fb: np.ndarray,
171     freq_fa: np.ndarray, mag_fa: np.ndarray,
172     freq_mx: np.ndarray, mag_mx: np.ndarray,
173     out: pathlib.Path,
174 ) -> None:
175     """Three-panel bare spectrum figure (no annotations)."""
176     fig = plt.figure(figsize=(10, 8))
177     gs = gridspec.GridSpec(3, 1, hspace=0.55)
178
179     # Panel 1: mains harmonics
180     ax1 = fig.add_subplot(gs[0])
181     ax1.plot(freq_m, mag_m, linewidth=0.9, color=COLORS["mains_line"])
182     ax1.set_xlim(0, 600)
183     ax1.set_xlabel("Frequency (Hz)")
184     ax1.set_ylabel("Magnitude")
185     ax1.set_title("1. AC mains spectrum (0-600 Hz)")
186     ax1.grid(alpha=0.3)
187
188     # Panel 2: low-pass filter before / after
189     ax2 = fig.add_subplot(gs[1])
190     ax2.plot(freq_fb, mag_fb, linewidth=0.9, color=COLORS["before"],
191             label="before filter", alpha=0.8)
192     ax2.plot(freq_fa, mag_fa, linewidth=1.1, color=COLORS["after"],
193             label="after filter", alpha=0.9)
194     ax2.set_xlim(0, 1_800)
195     ax2.set_xlabel("Frequency (Hz)")
196     ax2.set_ylabel("Magnitude")
197     ax2.set_title("2. Sallen-Key LP filter (fc = 300 Hz)")
198     ax2.legend(fontsize=8)
199     ax2.grid(alpha=0.3)
200
201     # Panel 3: mixer output
202     ax3 = fig.add_subplot(gs[2])
203     ax3.plot(freq_mx, mag_mx, linewidth=0.9, color=COLORS["mains_line"])
204     ax3.set_xlim(0, 2_200)
205     ax3.set_xlabel("Frequency (Hz)")
206     ax3.set_ylabel("Magnitude")
207     ax3.set_title("3. Heterodyne mixer output (RF x LO)")
208     ax3.grid(alpha=0.3)
209
210     fig.savefig(out, dpi=110)
211     plt.close(fig)
212
213
214 def fig_takeaway(
215     freq_m: np.ndarray, mag_m: np.ndarray,
216     freq_fb: np.ndarray, mag_fb: np.ndarray,
217     freq_fa: np.ndarray, mag_fa: np.ndarray,
218     freq_mx: np.ndarray, mag_mx: np.ndarray,
219     out: pathlib.Path,
220 ) -> None:
221     """Three-panel annotated takeaway figure."""

```



```

222 fig = plt.figure(figsize=(10, 9))
223 gs = gridspec.GridSpec(3, 1, hspace=0.65)
224
225 # ----- Panel 1: mains harmonics (annotated) -----
226 ax1 = fig.add_subplot(gs[0])
227 ax1.plot(freq_m, mag_m, linewidth=0.9, color=COLORS["mains_line"])
228 ax1.set_xlim(0, 600)
229
230 for i, (label, (f, _)) in enumerate(MAINS_HARMONICS.items()):
231     c = COLORS["harmonics"][i]
232     ax1.axvline(f, color=c, linestyle="--", linewidth=0.8, alpha=0.7)
233     # Amplitude from table (spectrum shows peak; for annotation use
234     # table value)
235     idx = int(round(f / FS * N))
236     idx = min(idx, mag_m.size - 1)
237     y = float(mag_m[idx])
238     offset_x = 8 if i % 2 == 0 else -70
239     ax1.annotate(label, xy=(f, y), xytext=(f + offset_x, y + 0.04),
240                 fontsize=8, color=c,
241                 arrowprops=dict(arrowstyle="->", color=c,
242                               linewidth=0.6))
243
244 thd_pct = thd(MAINS_HARMONICS) * 100.0
245 ax1.text(0.97, 0.92, f"THD = {thd_pct:.1f}%", transform=ax1.transAxes,
246         ha="right", va="top", fontsize=9, color="#500000",
247         bbox=dict(boxstyle="round,pad=0.3", facecolor="#fff0f0",
248                 alpha=0.8))
249 ax1.set_xlabel("Frequency (Hz)")
250 ax1.set_ylabel("Magnitude")
251 ax1.set_title("1. AC mains: fundamental + odd harmonics from
252             non-linear loads")
253 ax1.grid(alpha=0.3)
254
255 # ----- Panel 2: filter before/after (annotated) -----
256 ax2 = fig.add_subplot(gs[1])
257 ax2.plot(freq_fb, mag_fb, linewidth=0.9, color=COLORS["before"],
258         label="input (broadband)", alpha=0.7)
259 ax2.plot(freq_fa, mag_fa, linewidth=1.1, color=COLORS["after"],
260         label="filtered output", alpha=0.9)
261 ax2.axvline(FILTER_FC, color=COLORS["cutoff"], linestyle=":",
262            linewidth=1.2)
263 ax2.text(FILTER_FC + 15, ax2.get_ylim()[1] * 0.5 if ax2.get_ylim()[1]
264         > 0 else 0.4,
265         f"fc = {FILTER_FC:.0f} Hz\n-40 dB/decade", fontsize=8,
266         color=COLORS["cutoff"])
267 # Arrow pointing to -40 dB/decade rolloff region
268 ax2.annotate("content above fc\nattenuated here", xy=(800, 0.05),
269            xytext=(1100, 0.28), fontsize=8, color=COLORS["after"],
270            arrowprops=dict(arrowstyle="->", color=COLORS["after"],
271                            linewidth=0.7))
272 ax2.set_xlim(0, 1_800)
273 ax2.set_xlabel("Frequency (Hz)")
274 ax2.set_ylabel("Magnitude")
275 ax2.set_title("2. Filter = multiplier in frequency domain: content
276             above fc shrinks")
277 ax2.legend(fontsize=8)
278 ax2.grid(alpha=0.3)
279
280 # ----- Panel 3: mixer output (annotated) -----
281 ax3 = fig.add_subplot(gs[2])
282 ax3.plot(freq_mx, mag_mx, linewidth=0.9, color=COLORS["mains_line"])
283 ax3.set_xlim(0, 2_200)

```

```

277 # Mark RF, LO, IF-diff, IF-sum
278 annotations = [
279     (LO_HZ, "LO\n900 Hz", COLORS["lo"], +70),
280     (RF_HZ, "RF\n1000 Hz", COLORS["rf"], +70),
281     (RF_HZ - LO_HZ, "IF-\n100 Hz", COLORS["if_diff"], +30),
282     (RF_HZ + LO_HZ, "IF+\n1900 Hz", COLORS["if_sum"], -100),
283 ]
284 for f, lbl, c, dx in annotations:
285     ax3.axvline(f, color=c, linestyle="--", linewidth=0.9, alpha=0.7)
286     idx = int(round(f / FS * N))
287     idx = min(idx, mag_mx.size - 1)
288     y = float(mag_mx[idx])
289     ax3.annotate(lbl, xy=(f, y), xytext=(f + dx, y + 0.03),
290                 fontsize=8, color=c,
291                 arrowprops=dict(arrowstyle="->", color=c,
292                                 linewidth=0.7))
292 ax3.text(0.5, 0.88,
293         r"$f_{\text{RF}} \times f_{\text{LO}} \rightarrow (f_{\text{RF}} - f_{\text{LO}}) + (f_{\text{RF}} + f_{\text{LO}})$",
294         transform=ax3.transAxes, ha="center", fontsize=9,
295         bbox=dict(boxstyle="round, pad=0.3", facecolor="#f0f4ff",
296                 alpha=0.8))
296 ax3.set_xlabel("Frequency (Hz)")
297 ax3.set_ylabel("Magnitude")
298 ax3.set_title("3. Heterodyne: multiply RF x LO in time = spectrum shift in frequency")
299 ax3.grid(alpha=0.3)
300
301 fig.savefig(out, dpi=110)
302 plt.close(fig)
303
304
305 # ---- Main
306 -----
307 def main() -> int:
308     here = pathlib.Path(__file__).resolve().parent
309     default_out = here.parent.parent.parent / "docs" / "shad" / "figures"
310
311     parser = argparse.ArgumentParser(description=__doc__)
312     parser.add_argument("--out-dir", type=pathlib.Path,
313                         default=default_out)
314     args = parser.parse_args()
315     args.out_dir.mkdir(parents=True, exist_ok=True)
316
317     rng = np.random.default_rng(20260511)
318     t = make_time()
319
320     # -- Sub-topic 1: mains
321     -----
322     mains = synth_mains(t, rng)
323     freq_m, mag_m = spectrum(mains, window=False) # pure tones: no window needed
324
325     # -- Sub-topic 2: active filter
326     -----
327     filt_in = synth_filter_input(t, rng)
328     filt_out = apply_sk_lowpass(filt_in, fc=FILTER_FC, q=FILTER_Q)
329     freq_fb, mag_fb = spectrum(filt_in, window=True)
330     freq_fa, mag_fa = spectrum(filt_out, window=True)
331
332     # -- Sub-topic 3: heterodyne mixer
333     -----

```

```

330     _, _, mix = synth_mixer(t)
331     freq_mx, mag_mx = spectrum(mix, window=False)
332
333     # -- Render figures
334     -----
335     fig_input(t, mains, args.out_dir / "fig-b4-input.png")
336     fig_spectrum(freq_m, mag_m,
337                 freq_fb, mag_fb, freq_fa, mag_fa,
338                 freq_mx, mag_mx,
339                 args.out_dir / "fig-b4-spectrum.png")
340     fig_takeaway(freq_m, mag_m,
341                 freq_fb, mag_fb, freq_fa, mag_fa,
342                 freq_mx, mag_mx,
343                 args.out_dir / "fig-b4-takeaway.png")
344
345     print("B4 done.")
346     print(f" AC mains THD : {thd(MAINS_HARMONICS)*100:.1f}%")
347     print(f" Mixer IF-      : {RF_HZ - LO_HZ:.0f} Hz  (RF - LO)")
348     print(f" Mixer IF+      : {RF_HZ + LO_HZ:.0f} Hz  (RF + LO)")
349     print(f" Figures in      : {args.out_dir}")
350     return 0
351
352 # ---- Real-data extension notes
353 # -----
354 # AC mains: OPSP (Open Power System Data) publishes 50-Hz grid time-series.
355 # https://data.open-power-system-data.org/ (CC-BY-4.0)
356 # Load a 1-second window from any 50 Hz grid recording and run
357 # spectrum(data, window=False) -- the workflow above is identical.
358 #
359 # RF captures: rtl-sdr.com publishes sample IQ captures (public domain).
360 # Decode with scipy.io.wavfile and feed the real part to synth_mixer().
361 # The IF arithmetic is unchanged.
362
363 if __name__ == "__main__":
364     raise SystemExit(main())

```


C Complete Script: B5 Radar

Listing C.1: examples/shad/b5-radar/main.py — LFM chirp + matched filter + range-Doppler map

```
1  #!/usr/bin/env python3
2  # SPDX-License-Identifier: Apache-2.0
3  """
4  b5-radar/main.py - Shad-tier Band 5: Radar (NEXRAD-style)
5
6  Three progressive stages of the same Fourier machinery:
7
8  Stage 1 - Single chirp pulse + matched-filter pulse compression
9  A linear-FM (chirp) pulse is transmitted. The received echo is
10 delayed (target range) and buried in noise (SNR ~ 0 dB raw).
11 Cross-correlating the echo with the known transmit waveform in the
12 frequency domain (matched filter) compresses the pulse energy into
13 a sharp range peak, lifting SNR by the time-bandwidth product.
14
15 Stage 2 - Pulse train + Doppler DFT
16 64 pulses are transmitted at PRF = 1000 Hz. A moving target
17 imprints a phase shift between successive echoes proportional to
18 its radial velocity (Doppler shift:  $df = 2*v_r*f_c/c$ ).
19 A DFT across the pulse-train direction converts phase history
20 into velocity.
21
22 Stage 3 - 2-D range-Doppler map (NEXRAD-style volume scan)
23 Stack all 64 compressed range profiles into a matrix:
24     rows = range bins (fast-time, within a pulse)
25     cols = pulse index (slow-time, across the CPI)
26 Apply DFT along slow-time axis -> range-Doppler map.
27 Three targets at different ranges and velocities appear as
28 isolated bright cells in a 2-D heatmap.
29
30 Parameters mirror a real NEXRAD S-band unit (WSR-88D long-pulse mode):
31     f_c = 2.8 GHz (S-band carrier)
32     tau = 1.6 us (pulse duration)
33     BW = 0.625 MHz (chirp bandwidth; time-bandwidth product = 1.0)
34     PRF = 1000 Hz (unambiguous range ~ 150 km, unamb velocity ~ 27 m/s)
35     N_p = 64 (pulses per coherent processing interval)
36     fs = 2 MHz (fast-time sample rate within a pulse)
37
38 Targets (synthesised):
39 T1: range 30 km, radial velocity +20 m/s (precipitation, approaching)
40 T2: range 65 km, radial velocity 0 m/s (ground clutter analogue)
41 T3: range 110 km, radial velocity -35 m/s (precipitation, receding)
42
43 Reproducibility seed: 20260512 (figures are byte-reproducible on re-runs)
44
45 Output (relative to fourier/docs/shad/figures/):
46     fig-b5-input.png          time-domain: transmit chirp + noisy echo (stage 1)
```

```

47 fig-b5-spectrum.png    bare 2-D range-Doppler heatmap (stage 3)
48 fig-b5-takeaway.png    annotated range-Doppler map (all targets labelled)
49 """
50 from __future__ import annotations
51
52 import argparse
53 import pathlib
54
55 import matplotlib
56 matplotlib.use("Agg")
57 import matplotlib.pyplot as plt
58 import matplotlib.gridspec as gridspec
59 import numpy as np
60
61 # -- Reproducibility
62 -----
63 RNG_SEED = 20260512
64
65 # -- Radar system parameters (NEXRAD WSR-88D long-pulse analogue)
66 -----
67 C_LIGHT    = 3.0e8          # speed of light (m/s)
68 F_C        = 2.8e9          # S-band carrier (Hz)
69 TAU_S      = 1.6e-6         # pulse duration (s)
70 BW_HZ      = 0.625e6        # chirp bandwidth (Hz); TBP = BW * tau = 1.0
71 FS_HZ      = 2.0e6          # fast-time sample rate (Hz; = 2x chirp BW)
72 PRF_HZ     = 1_000.0        # pulse repetition frequency (Hz)
73 N_PULSES   = 64             # coherent processing interval (CPI) length
74
75 # Derived
76 LAMBDA     = C_LIGHT / F_C   # wavelength ~ 10.7 cm
77 T_PRI      = 1.0 / PRF_HZ    # pulse repetition interval (s)
78 R_UNAMBIG  = C_LIGHT * T_PRI / 2.0 # max unambiguous range ~ 150 km
79 V_UNAMBIG  = PRF_HZ * LAMBDA / 4.0 # max unambiguous speed ~ 26.8
80                                     m/s
81 N_SAMP_PU  = int(FS_HZ * TAU_S) # samples per pulse = 3
82 N_RANGE    = int(FS_HZ * T_PRI) # samples per PRI = 2000
83 CHIRP_K    = BW_HZ / TAU_S     # chirp rate (Hz/s)
84
85 # -- Targets
86 -----
87 # (range_m, radial_velocity_m_s, relative_RCS)
88 TARGETS = [
89     (30_000.0, +20.0, 1.00), # T1: approaching precipitation cell
90     (65_000.0, 0.0, 0.50),  # T2: stationary (ground clutter analogue)
91     (110_000.0, -35.0, 0.70), # T3: receding precipitation cell
92 ]
93
94 # -- SNR budget
95 -----
96 # Raw single-pulse SNR per target (before compression or integration):
97 SINGLE_PULSE_SNR_DB = 0.0 # 0 dB -- echo buried in noise
98
99 # After matched-filter compression: gain = TBP = BW*tau = 1.0 (our BW is
100 tiny;
101 # in a real NEXRAD super-resolution mode BW ~ 0.625 MHz gives TBP >> 1).
102 # After N_PULSES integration: gain = 10*log10(64) ~ 18 dB.
103 # Total expected SNR at output: ~ 18 dB -- clearly visible above noise
104 floor.
105
106 #
107
108 # SYNTHESIS

```

```

101 #
102     ?????????????????????????????????????????????????????????????????????????????????????
103 def make_chirp() -> np.ndarray:
104     """
105     Build the complex baseband transmit chirp.
106
107     A linear-FM (LFM) chirp sweeps from -BW/2 to +BW/2 over tau seconds:
108         s_tx(t) = exp(j * pi * K * t^2),    0 <= t < tau
109     where K = BW / tau is the chirp rate.
110
111     In the frequency domain, a chirp has near-constant magnitude across BW
112     and a quadratic phase. The matched filter reverses this phase,
113     producing
114     a sinc-like compressed pulse whose 3-dB width is 1/BW.
115     """
116     t = np.arange(N_SAMP_PU) / FS_HZ          # time axis within the pulse
117     # Complex baseband LFM chirp (equation LFM-1 in signal-processing
118     # literature)
119     chirp = np.exp(1j * np.pi * CHIRP_K * t ** 2)
120     return chirp.astype(np.complex128)
121
122 def range_delay_samples(range_m: float) -> int:
123     """Convert range (metres) to round-trip delay in fast-time samples."""
124     tau_rt = 2.0 * range_m / C_LIGHT          # round-trip travel time
125     return int(np.round(tau_rt * FS_HZ))      # convert to sample index
126
127 def doppler_phase_per_pulse(vel_m_s: float) -> float:
128     """
129     Phase advance (radians) between successive pulses for a target moving
130     at radial velocity v_r (positive = approaching radar).
131
132     Each PRI the target moves v_r * T_PRI metres closer. The two-way path
133     shortens by 2 * v_r * T_PRI metres, adding a phase:
134         phi = 2*pi * (2 * v_r * T_PRI) / lambda
135             = 2*pi * v_r / (PRF * lambda / 2)
136             = 2*pi * f_d / PRF
137     where f_d = 2*v_r*f_c/c is the Doppler frequency.
138     """
139     f_doppler = 2.0 * vel_m_s * F_C / C_LIGHT
140     return 2.0 * np.pi * f_doppler / PRF_HZ
141
142 def build_pulse_matrix(chirp: np.ndarray, rng: np.random.Generator) ->
143     np.ndarray:
144     """
145     Build the full N_RANGE x N_PULSES raw receive matrix.
146
147     For each of N_PULSES transmissions, each target contributes a delayed,
148     Doppler-shifted, amplitude-scaled echo of the chirp. White complex
149     Gaussian noise is added to achieve the target single-pulse SNR.
150
151     Returns shape: (N_RANGE, N_PULSES) complex128
152     """
153     # Signal amplitude from SNR target:
154     # SNR = A^2 / noise_power; noise_power = sigma^2 per complex sample
155     noise_sigma = 1.0                      # normalise noise to unit
156     # power
157     sig_amp_per_rcs1 = noise_sigma * 10 ** (SINGLE_PULSE_SNR_DB / 20.0)
158     matrix = np.zeros((N_RANGE, N_PULSES), dtype=np.complex128)

```

```

159
160     for pulse_idx in range(N_PULSES):
161         # Add echo from each target
162         for (r_m, v_mps, rcs) in TARGETS:
163             delay      = range_delay_samples(r_m)
164             dphi       = doppler_phase_per_pulse(v_mps)
165             amplitude  = sig_amp_per_rcs1 * np.sqrt(rcs)
166             phase      = pulse_idx * dphi          # slow-time phase history
167
168             # Inject chirp at delay offset (zero-pad if needed)
169             end = delay + N_SAMP_PU
170             if end <= N_RANGE:
171                 matrix[delay:end, pulse_idx] += amplitude * np.exp(1j *
172                     phase) * chirp
173
174             # Additive white complex Gaussian noise
175             noise = rng.normal(0, noise_sigma / np.sqrt(2), N_RANGE)
176             noise = noise + 1j * rng.normal(0, noise_sigma / np.sqrt(2),
177                 N_RANGE)
178             matrix[:, pulse_idx] += noise
179
180     return matrix
181
182 #
183 #
184
185 def matched_filter_compress(raw_matrix: np.ndarray, chirp: np.ndarray) ->
186     np.ndarray:
187     """
188     Stage 1+2: Apply matched filter along the fast-time (range) axis.
189
190     The matched filter for signal s(t) is its time-reversed conjugate
191     s*(-t).
192     In the frequency domain, cross-correlation = pointwise multiply by
193     conj(S(f)):
194
195     compressed(t) = IFFT( FFT(rx) * conj(FFT(tx)) )
196
197     This is the same DFT we have been using throughout the Shad tier,
198     now used for extraction rather than description.
199
200     Parameters
201     -----
202     raw_matrix : shape (N_RANGE, N_PULSES)
203     chirp      : shape (N_SAMP_PU,) -- the known transmit waveform
204
205     Returns
206     -----
207     compressed : shape (N_RANGE, N_PULSES)
208         Each column is one pulse's range profile after compression.
209     """
210     N_FFT = N_RANGE          # FFT length = full PRI length
211
212     # Pre-compute the reference spectrum (zero-padded to N_FFT)
213     chirp_padded = np.zeros(N_FFT, dtype=np.complex128)
214     chirp_padded[:N_SAMP_PU] = chirp
215     TX = np.fft.fft(chirp_padded)    # transmit reference spectrum
216     TX_CONJ = np.conj(TX)            # conjugate = matched filter kernel

```



```

215     compressed = np.zeros_like(raw_matrix)
216     for p in range(N_PULSES):
217         RX = np.fft.fft(raw_matrix[:, p], n=N_FFT)    # receive spectrum
218         # Multiply: correlation in time = conjugate-product in frequency
219         compressed[:, p] = np.fft.ifft(RX * TX_CONJ).real
220
221     return compressed
222
223
224 def range_doppler_map(compressed: np.ndarray) -> np.ndarray:
225     """
226     Stage 3: Compute range-Doppler map via slow-time DFT.
227
228     After pulse compression, each row of 'compressed' is a range profile.
229     A DFT across the N_PULSES dimension of each range bin extracts the
230     Doppler frequency of any target present at that range:
231
232         RD[range_bin, doppler_bin] = DFT_slow_time( compressed[range_bin,
233             :])
234
235     The Doppler axis spans -PRF/2 to +PRF/2 (after fftshift),
236     which maps to radial velocities -V_unambig to +V_unambig.
237
238     Parameters
239     -----
240     compressed : shape (N_RANGE, N_PULSES)
241
242     Returns
243     -----
244     rd_map : shape (N_RANGE, N_PULSES), magnitude of the Doppler DFT,
245             with Doppler axis zero-centred (fftshift applied).
246     """
247     # Apply Hann window along slow-time to suppress Doppler sidelobes
248     window = np.hanning(N_PULSES)    # shape
249         (N_PULSES,)
250     windowed = compressed * window[np.newaxis, :]    # broadcast over
251         range
252
253     # DFT across slow-time (axis=1)
254     rd_complex = np.fft.fft(windowed, axis=1)    # shape (N_RANGE,
255         N_PULSES)
256
257     # fftshift to centre zero-Doppler
258     rd_shifted = np.fft.fftshift(rd_complex, axes=1)
259
260     return np.abs(rd_shifted)
261
262 #
263     ?????????????????????????????????????????????????????????????????????????????????????????????
264 # AXIS UTILITIES
265 #
266     ?????????????????????????????????????????????????????????????????????????????????????????????
267
268 def range_axis_km() -> np.ndarray:
269     """Range axis in kilometres: 0 to R_unambig."""
270     return np.arange(N_RANGE) * C_LIGHT / (2.0 * FS_HZ) / 1_000.0
271
272 def velocity_axis_ms() -> np.ndarray:
273     """
274     Doppler velocity axis (m/s), zero-centred after fftshift.
275     Bin spacing = PRF / N_PULSES. Range = [-V_unambig, +V_unambig].

```

```

272     """
273     f_bins = np.fft.fftshift(np.fft.fftfreq(N_PULSES, d=1.0 / PRF_HZ))
274     return f_bins * LAMBDA / 2.0      # frequency -> radial velocity
275
276
277 #
278 # FIGURES
279 #
280
281 COLORS = {
282     "tx":      "#1f4e79",
283     "rx":      "#b04040",
284     "target":  ["#22882e", "#c07000", "#b02020"],
285 }
286
287
288 def fig_input(chirp: np.ndarray, raw_matrix: np.ndarray,
289              out: pathlib.Path) -> None:
290     """
291     Two-panel time-domain figure:
292     Panel 1: transmit chirp (real part; shows FM sweep)
293     Panel 2: received echo for pulse 0, first 300 samples -- shows the
294             noise floor and the faint echo from T1 at delay ~ 200 samples.
295     """
296     t_pulse_us = np.arange(N_SAMP_PU) / FS_HZ * 1e6      # microseconds
297
298     # Single-pulse receive signal (pulse 0), converted to time axis in us
299     t_rx_us = np.arange(N_RANGE) / FS_HZ * 1e6
300     rx_real = raw_matrix[:, 0].real
301
302     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 5.5))
303     fig.suptitle("B5 - Radar: transmit chirp and noisy received echo",
304                  fontsize=11)
305
306     # Panel 1: transmit chirp
307     ax1.plot(t_pulse_us, chirp.real, lw=1.0, color=COLORS["tx"],
308              label="Re{tx chirp}")
309     ax1.plot(t_pulse_us, chirp.imag, lw=0.7, color="#4090c0", alpha=0.6,
310              label="Im{tx chirp}", linestyle="--")
311     ax1.set_xlabel("Time within pulse (us)")
312     ax1.set_ylabel("Amplitude")
313     ax1.set_title(f"Transmit: linear-FM chirp, BW={BW_HZ/1e6:.3f} MHz, "
314                  f"f?={TAU_S*1e6:.1f} us")
315     ax1.legend(fontsize=8, loc="upper right")
316     ax1.grid(alpha=0.3)
317
318     # Panel 2: received echo (first 400 samples ? first 200 us ? 0-30 km
319     # range)
320     show_n = 400
321     ax2.plot(t_rx_us[:show_n], rx_real[:show_n], lw=0.6,
322              color=COLORS["rx"], label="Re{rx echo} -- pulse 0")
323     # Mark where T1's echo is expected
324     d1 = range_delay_samples(TARGETS[0][0])
325     ax2.axvspan(d1 / FS_HZ * 1e6, (d1 + N_SAMP_PU) / FS_HZ * 1e6,
326                 alpha=0.18, color="#22882e", label=f"T1 echo window (range
327                 {TARGETS[0][0]/1e3:.0f} km)")
328     ax2.set_xlabel("Fast time (us) [= round-trip range / c x 106]")
329     ax2.set_ylabel("Amplitude")
330     ax2.set_title(f"Received echo: SNR ? {SINGLE_PULSE_SNR_DB:.0f} dB raw
331                  -- "
332                  "echo buried in noise (first 200 us shown)")

```

```

328     ax2.legend(fontsize=8, loc="upper right")
329     ax2.grid(alpha=0.3)
330
331     fig.tight_layout()
332     fig.savefig(out, dpi=110)
333     plt.close(fig)
334
335
336 def fig_spectrum(rd_map: np.ndarray, range_km: np.ndarray,
337                 vel_ms: np.ndarray, out: pathlib.Path) -> None:
338     """
339     Bare range-Doppler heatmap (log magnitude, dB scale).
340     Rows = range bins (km), columns = Doppler bins (m/s).
341     Uses imshow with extent set to physical axes.
342     """
343     rd_db = 20.0 * np.log10(rd_map + 1e-12) # avoid log(0)
344     # Crop to interesting range/velocity window
345     r_max_idx = int(130_000 / (C_LIGHT / (2.0 * FS_HZ))) # 130 km
346     rd_crop = rd_db[:r_max_idx, :]
347     r_axis = range_km[:r_max_idx]
348
349     vmin = np.percentile(rd_crop, 5)
350     vmax = np.percentile(rd_crop, 99.5)
351
352     fig, ax = plt.subplots(figsize=(9, 5))
353     im = ax.imshow(rd_crop, aspect="auto", origin="lower",
354                   extent=[vel_ms[0], vel_ms[-1], r_axis[0], r_axis[-1]],
355                   cmap="turbo", vmin=vmin, vmax=vmax,
356                   interpolation="nearest")
357     cb = fig.colorbar(im, ax=ax, shrink=0.85)
358     cb.set_label("Magnitude (dB)", fontsize=9)
359     ax.set_xlabel("Radial velocity (m/s) [negative = receding]")
360     ax.set_ylabel("Range (km)")
361     ax.set_title("B5 - Range-Doppler map (bare): 2-D DFT over 64 pulses")
362     ax.grid(alpha=0.2, color="white", lw=0.4)
363     fig.tight_layout()
364     fig.savefig(out, dpi=110)
365     plt.close(fig)
366
367 def fig_takeaway(rd_map: np.ndarray, range_km: np.ndarray,
368                 vel_ms: np.ndarray, out: pathlib.Path) -> None:
369     """
370     Annotated range-Doppler map: each target cell labelled, wind-gradient
371     region highlighted, PRF-fold ambiguity zone marked.
372     """
373     rd_db = 20.0 * np.log10(rd_map + 1e-12)
374     r_max_idx = int(130_000 / (C_LIGHT / (2.0 * FS_HZ)))
375     rd_crop = rd_db[:r_max_idx, :]
376     r_axis = range_km[:r_max_idx]
377
378     vmin = np.percentile(rd_crop, 5)
379     vmax = np.percentile(rd_crop, 99.5)
380
381     fig, ax = plt.subplots(figsize=(10, 6))
382     ax.imshow(rd_crop, aspect="auto", origin="lower",
383             extent=[vel_ms[0], vel_ms[-1], r_axis[0], r_axis[-1]],
384             cmap="turbo", vmin=vmin, vmax=vmax, interpolation="nearest")
385
386     # Annotate each target
387     ann_params = [
388         # (true range km, true vel m/s, label, text_offset)

```

```

389         (30.0, +20.0, "T1: precip cell\n30 km, +20 m/s\n(approaching)",
390             (+2, +8)),
391         (65.0, 0.0, "T2: ground clutter\n65 km, 0 m/s",
392             (+2, +10)),
393         (110.0, -35.0, "T3: precip cell\n110 km, -35 m/s\n(receding)",
394             (+2, -15)),
395     ]
396     for r_km, v_ms, lbl, (dv, dr) in ann_params:
397         ax.annotate(lbl,
398                     xy=(v_ms, r_km),
399                     xytext=(v_ms + dv, r_km + dr),
400                     fontsize=8, color="white",
401                     arrowprops=dict(arrowstyle="->", color="white",
402                                     lw=0.9),
403                     bbox=dict(boxstyle="round,pad=0.2",
404                             facecolor="#00000080", edgecolor="none"))
405
406     # Mark zero-Doppler line (clutter ridge)
407     ax.axvline(0, color="#ffffff80", lw=0.9, linestyle="--")
408     ax.text(0.5, 118, "zero-Doppler\n(stationary clutter)", color="white",
409             fontsize=7.5, ha="left",
410             bbox=dict(boxstyle="round,pad=0.15", facecolor="#00000070",
411                     edgecolor="none"))
412
413     # Mark Doppler unambiguous limits
414     v_u = V_UNAMBIG
415     for sign, label in [(+1, f"+V_unamb\n+{v_u:.1f} m/s"),
416                        (-1, f"-V_unamb\n-{v_u:.1f} m/s")]:
417         ax.axvline(sign * v_u, color="yellow", lw=0.7, linestyle=":",
418                   alpha=0.6)
419         ax.text(sign * v_u + 0.4, 5, label, color="yellow",
420               fontsize=6.5, ha="center", va="bottom",
421               bbox=dict(boxstyle="round,pad=0.1",
422                       facecolor="#00000080", edgecolor="none"))
423
424     ax.set_xlabel("Radial velocity (m/s) [negative = receding from
425                  radar]",
426                  fontsize=9)
427     ax.set_ylabel("Range (km)", fontsize=9)
428     ax.set_title("B5 takeaway - reading the range-Doppler map like a radar
429                  engineer",
430                  fontsize=10)
431     ax.grid(alpha=0.15, color="white", lw=0.4)
432     fig.tight_layout()
433     fig.savefig(out, dpi=110)
434     plt.close(fig)
435
436 #
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

443
444     rng = np.random.default_rng(RNG_SEED)
445     chirp = make_chirp()
446
447     print("Building raw pulse matrix (64 pulses x 2000 fast-time
         samples)...")
448     raw = build_pulse_matrix(chirp, rng)
449
450     print("Applying matched-filter pulse compression (range DFT +
         conjugate multiply)...")
451     compressed = matched_filter_compress(raw, chirp)
452
453     print("Computing range-Doppler map (slow-time DFT over 64 pulses)...")
454     rd_map = range_doppler_map(compressed)
455     range_km = range_axis_km()
456     vel_ms = velocity_axis_ms()
457
458     # Verify targets are visible: find peak in each target region
459     print("\nTarget detection summary:")
460     for label, (r_m, v_mps, rcs) in zip(["T1", "T2", "T3"], TARGETS):
461         r_bin = range_delay_samples(r_m)
462         f_d = 2.0 * v_mps * F_C / C_LIGHT
463         v_bin_raw = int(np.round(f_d / PRF_HZ * N_PULSES))
464         v_bin = (N_PULSES // 2 + v_bin_raw) % N_PULSES # after fftshift
465         window = 5
466         r_sl = slice(max(0, r_bin - window), min(N_RANGE, r_bin + window))
467         v_sl = slice(max(0, v_bin - window), min(N_PULSES, v_bin + window))
468         peak = rd_map[r_sl, v_sl].max()
469         noise_floor = np.median(rd_map)
470         snr_est = 20 * np.log10(peak / (noise_floor + 1e-12))
471         print(f" {label}: range {r_m/1e3:.0f} km, vel {v_mps:+.0f} m/s "
              f"-> estimated peak SNR {snr_est:.1f} dB")
472
473
474     print(f"\nSystem parameters:")
475     print(f" Unambiguous range: {R_UNAMBIG/1e3:.0f} km")
476     print(f" Unambiguous velocity: +/-{V_UNAMBIG:.1f} m/s")
477     print(f" Range resolution: {C_LIGHT/(2*BW_HZ):.0f} m (= c /
         2BW)")
478     print(f" Velocity resolution: {PRF_HZ*LAMBDA/2/N_PULSES*100:.0f}
         cm/s (= PRF*lambda/2/N_pulses)")
479
480     print("\nRendering figures...")
481     fig_input(chirp, raw, args.out_dir / "fig-b5-input.png")
482     fig_spectrum(rd_map, range_km, vel_ms, args.out_dir /
         "fig-b5-spectrum.png")
483     fig_takeaway(rd_map, range_km, vel_ms, args.out_dir /
         "fig-b5-takeaway.png")
484     print(f"Figures saved to: {args.out_dir}")
485     return 0
486
487
488 # -- Real-data extension notes
489 # -----
490 # NEXRAD Level-II data is public domain (NOAA / AWS Open Data):
491 # https://registry.opendata.aws/noaa-nexrad/
492 # Format: bzip2-compressed binary; use PyART (pip install arm_pyart) or
493 # wradlib (pip install wradlib) to read Level-II volumes.
494 # The Doppler velocity field is pre-computed by the WSR-88D unit; you can
495 # also work from the raw I/Q if your dataset includes it (Level-2.5
496 # mode).
497 # The matched-filter and range-Doppler pipeline above is structurally
498 # identical to what the WSR-88D's signal processor does, just at 2.8 GHz
499 # instead of our baseband 2 MHz simulation.

```

```
498
499 if __name__ == "__main__":
500     raise SystemExit(main())
```

D Canonical Equations Reference

The three equations this project is built on, in L^AT_EX form as stored in `fourier/shared/canonical-equations`

DFT-1 (Discrete Fourier Transform):

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-2\pi i k n / N}$$

DFT-2 (Inverse DFT):

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot e^{+2\pi i k n / N}$$

FFT-1 (Cooley-Tukey radix-2 butterfly):

$$X[k] = X_{\text{even}}[k] + \omega_N^k \cdot X_{\text{odd}}[k], \quad X[k + N/2] = X_{\text{even}}[k] - \omega_N^k \cdot X_{\text{odd}}[k]$$

PS-1 (Fourier partial sum):

$$S_N(x) = \sum_{k=-N}^N c_k e^{ikx}, \quad c_k = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) e^{-ikx} dx$$

Colophon

Figures generated by Python 3 with NumPy and Matplotlib.

No SciPy used — all DFT computations via `np.fft.fft` (same algorithm as the Fortran and C++ reference kernels in the repo).

Typeset with L^AT_EX (TeX Live 2022), Computer Modern font, `tcolorbox`.

Dragon icon generated programmatically from matplotlib patches (`fourier/docs/shad/figures/make_drago`).

Reproducibility seeds: B1 = 42, B4 = 20260511, B5 = 20260512.

All code Apache-2.0. All documentation CC-BY-SA-4.0.

Repo: <https://github.com/lege-artis/fourier>